



Lab 5: Mixing with C

Reference

- MPLAB® XC8 C Compiler User’s Guide

(https://ww1.microchip.com/downloads/aemDocuments/documents/DEV/ProductDocuments/UserGuides/MPLAB-XC8-C-Compiler-Users-Guide-for-PIC-DS50002737.pdf)

- MPLAB® XC8 PIC® Assembler User's Guide

(https://ww1.microchip.com/downloads/en/DeviceDoc/MPLAB%20XC8%20PIC%20Assembler%20User%27s%20Guide%2050002974A.pdf)

Data Types

詳細請參考 XC8 C Compiler UG 5.3

(https://ww1.microchip.com/downloads/aemDocuments/documents/DEV/ProductDocuments/UserGuides/MPLAB-XC8-C-Compiler-Users-Guide-for-PIC-DS50002737.pdf#page=83)

Table 5-1. Integer Data Types

Type	Size (bits)	Arithmetic Type
`__bit`	1	Unsigned integer
`signed char`	8	Signed integer
`unsigned char`	8	Unsigned integer
`signed short`	16	Signed integer
`unsigned short`	16	Unsigned integer
`signed int`	16	Signed integer
`unsigned int`	16	Unsigned integer
`__int24`	24	Signed integer
`__uint24`	24	Unsigned integer
`signed long`	32	Signed integer
`unsigned long`	32	Unsigned integer
`signed long long`	32/64	Signed integer
`unsigned long long`	32/64	Unsigned integer

Function Types

根據 User's Guide

(https://ww1.microchip.com/downloads/aemDocuments/documents/DEV/ProductDocuments/UserGuides/MPLAB-XC8-C-Compiler-Users-Guide-for-PIC-DS50002737.pdf) 在 **<5.3.2> Integer Data Types** 中提及:

- `char` 為 8-bit

- `int` 為 16-bit
- `long` 為 32-bit
- `long long` 為 64-bit

Function Parameters

根據 User's Guide (https://ww1.microchip.com/downloads/aemDocuments/documents/DEV/ProductDocuments/UserGuides/MPLAB-XC8-C-Compiler-Users-Guide-for-PIC-DS50002737.pdf) 在 **<5.8.5> Function Parameters** 中提及：

- 若第一個參數大小 ≤ 1 Byte:
  該參數會放入 **WREG**;其他參數則依照各自的型別大小與宣告順序，依序配置在該函式的 Parameter Memory(亦即 File Registers)，並自 `0x001` 起連續分配 (`0x001`、`0x002`、...) 向後遞增。
- 若第一個參數大小 > 1 Byte: 所有參數依其型別大小與宣告順序，自 Parameter Memory 的 `0x001` 起連續配置，後續參數緊接其後分配。

> 以上都要遵循 XC8 所使用之 **Little Endian** 存放方式。

Function Return Values

根據 User's Guide (https://ww1.microchip.com/downloads/aemDocuments/documents/DEV/ProductDocuments/UserGuides/MPLAB-XC8-C-Compiler-Users-Guide-for-PIC-DS50002737.pdf) 在 **<5.8.6> Function Return Values** 中提及:

XC8 會在函式回傳前，重複利用該函式的 Parameter Memory 作為回傳值的儲存區；因為此時參數已完成用途，該區域不再承載參數資料。

- 若回傳型別 ≤ 1 Byte: 以 **WREG** 回傳其值，且同時會將 **WREG** 的值寫入 Parameter Memory 的位址 `0x001`。
- 若回傳型別 > 1 Byte: 以 Parameter Memory 回傳其值，以 `0x001` 為起點依 **Little Endian** 組成回傳值。

對於 PIC18，當回傳值大小大於 4 Bytes 且小於 8 Bytes 時，XC8 還會將該 Parameter Memory 的位址載入 FSR0，以便呼叫方以 Indirect Addressing 的方式存取回傳資料。

Example

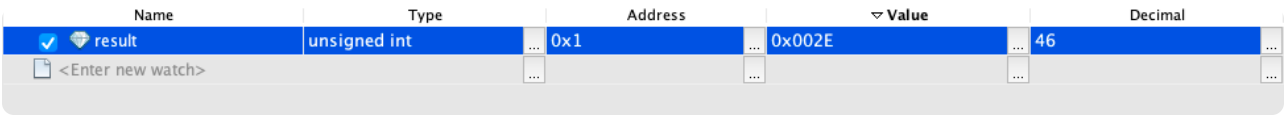
main.c

```
#include <xc.h>
    extern unsigned int add(unsigned int a, unsigned int b);
    void main(void){
        volatile unsigned int result = add(12, 34);
        while(1);
        return;
    }
```

add.asm

```
#include "xc.inc"
GLOBAL _add
PSECT mytext, local, class=CODE, reloc=2

_add:
      MOVFF 0x001, LATD
      MOVFF 0x003, LATC
      MOVFF LATD, WREG
      ADDWF 0x003, W
      MOVFF WREG, 0x001
      RETURN
```



> 更多細節可以參考 User's Guide (https://ww1.microchip.com/downloads/aemDocuments/documents/DEV/ProductDocuments/UserGuides/MPLAB-XC8-C-Compiler-Users-Guide-for-PIC-DS50002737.pdf) Guide
>
> - <3.3.1.4> How Can I Use A Variable Defined In Another Source File?
> - <3.3.1.5> How Can I Use A Function Defined In Another Source File?
> - <3.3.5.2> How Do I Stop An Unused Function Being Removed?
> - <5.3.8.2> Volatile Type Qualifier
> - <5.12.3.3> Equivalent Assembly Symbols
> - <6.1.9.22> Global Directive
> - <6.1.9.40> Psect Directive

Judge Information

Judge link: https://mclab-5.chih-hao.xyz/ (https://mclab-5.chih-hao.xyz/)

The account used for Judge is your student ID in uppercase letters. The password has been posted on Moodle.



If you cannot log in to Judge, please first try not to copy from Moodle, and enter the password manually. If you still cannot log in to Judge, please contact this TA:



!IMPORTANT! How to write the code

Since we will use a harness to test your code, it may use more memory during the process. To avoid incorrect results caused by modifications to unexpected registers during execution, **do not hardcode any register addresses that you use in your code (whether in parameters or in any variables you define yourself).**

Get parameters

To get the parameter of a function with more than one parameters, plase use `<function_name>$<parameter_number>` . In this task, you should use `_count_primes$0` and `_count_primes$1` instead of fixed register address to get the start and end of the range.

Define a variable

The correct approach of define variable is to use `PSECT` to reserve variable addresses in memory, so that the compiler automatically allocates unused memory space for them during linking, thereby ensuring that your program will not modify any memory space that has already been occupied during execution and cause unexpected results.

Here are some examples:

- Example 1: Reserve space in access bank by PSECT
  - **Wrong**

```
PSECT code, local, class=CODE, reloc=2
MOVLW    0x99
MOVWF    0x01, a
```

or

```
temp_var EQU 0x01

PSECT code, local, class=CODE, reloc=2
MOVLW    0x99
MOVWF    temp_var, a
```

or

```
#define temp_var 0x01

PSECT code, local, class=CODE, reloc=2
MOVLW    0x99
MOVWF    temp_var, a
```



- **Correct**

```
PSECT udata_acs
temp_var: DS 1

PSECT code, local, class=CODE, reloc=2
MOVLW    0x99
MOVWF    temp_var, a
```

- Example 2: Define a multi-bytes variable

```
#define int_variable_l int_variable
#define int_variable_h (int_variable+1)

PSECT udata_acs
int_variable: DS 2

PSECT code, local, class=CODE, reloc=2
MOVLW    0x99
MOVWF    int_variable_l, a
MOVLW    0x88
MOVWF    int_variable_h, a
```

or

```
int_variable_l EQU int_variable
int_variable_h EQU (int_variable+1)

PSECT udata_acs
int_variable: DS 2

PSECT code, local, class=CODE, reloc=2
MOVLW    0x99
MOVWF    int_variable_l, a
MOVLW    0x88
MOVWF    int_variable_h, a
```

- Example 3: Define a variable use in multiple files

`division.S`

```
GLOBAL div1, div2, divres

PSECT udata_acs
div1:   DS 1
div2:   DS 1
divres: DS 1
```

`main.S`

```
EXTRN div1, div2, divres

PSECT code, local, class=CODE, reloc=2
; divisio code...
MOVWF    divres, a
RETURN
```

> ⚠️ In the file where the variable is defined, use `GLOBAL` to export the variable, and where the variable is to be used, use `EXTRN` to import it. **Do not** directly include that file; otherwise, the variable will be reserved multiple times, causing its actual location to differ across files. However, if you define the variable in a `.inc` file, remember to include it in **one** of the files.

How to submit your code

Please compress your code into a `.zip` file before uploading it, and do not include the main function. For example, this is your project:



```
└─ your_project
   ├─ main.c
   ├─ count_primes.S
   ├─ division.inc
   ├─ division.S
   └─ your other files...
```

You need to put `count_primes.S`, `division.inc` and `division.S` into the zip file, but do not include `main.c`.

> 10/11 Update: Now you can upload a single or multiple files without compress.

About the score

You can modify the code and submit multiple times to try to get a better score. The score is calculated as follows:

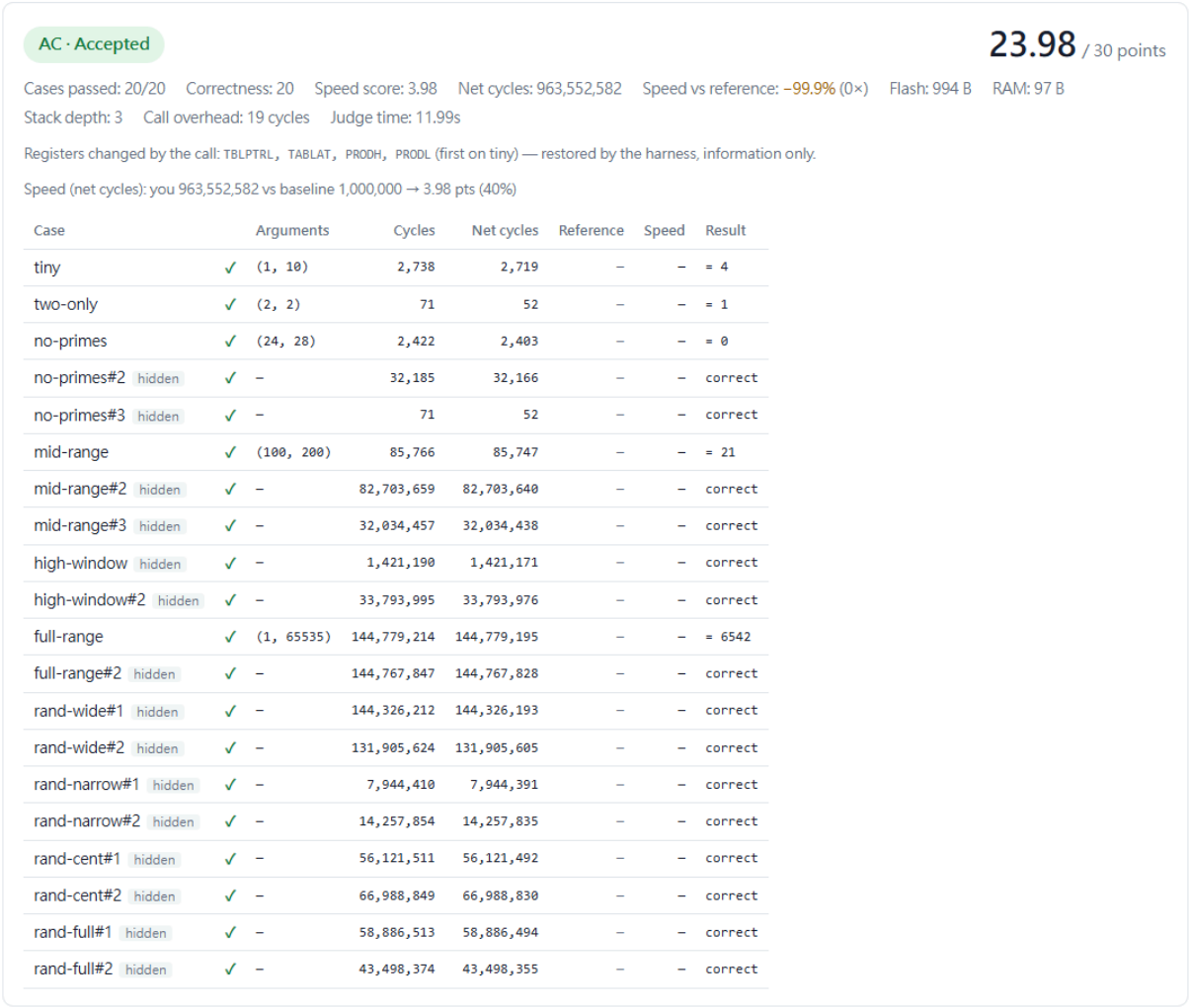
- There are 14 fixed test cases and 6 random test cases, for a total of 20 test cases. You get 1 point for each test case you answer correctly.

- After passing all test cases, your score will be calculated based on speed, using the following formula:

$$Score = 10 \times (1 - \frac{R - 1}{1600 - 1})$$

  R is the runtime of your program compared to the reference program. We will use the number of instruction cycles instead of the simulator's runtime for comparison, in order to avoid deviations caused by system load. The cycles of reference program is about ~~898.6M~~ ~~20K~~ 1M.

In the following example:



AC · Accepted

23.98 / 30 points

Cases passed: 20/20 Correctness: 20 Speed score: 3.98 Net cycles: 963,552,582 Speed vs reference: −99.9% (0×) Flash: 994 B RAM: 97 B
Stack depth: 3 Call overhead: 19 cycles Judge time: 11.99s

Registers changed by the call: TBLPTRL, TABLAT, PRODH, PRODL (first on tiny) — restored by the harness, information only.

Speed (net cycles): you 963,552,582 vs baseline 1,000,000 → 3.98 pts (40%)

Case		Arguments	Cycles	Net cycles	Reference	Speed	Result
tiny	✓	(1, 10)	2,738	2,719	–	–	= 4
two-only	✓	(2, 2)	71	52	–	–	= 1
no-primes	✓	(24, 28)	2,422	2,403	–	–	= 0
no-primes#2 hidden	✓	–	32,185	32,166	–	–	correct
no-primes#3 hidden	✓	–	71	52	–	–	correct
mid-range	✓	(100, 200)	85,766	85,747	–	–	= 21
mid-range#2 hidden	✓	–	82,703,659	82,703,640	–	–	correct
mid-range#3 hidden	✓	–	32,034,457	32,034,438	–	–	correct
high-window hidden	✓	–	1,421,190	1,421,171	–	–	correct
high-window#2 hidden	✓	–	33,793,995	33,793,976	–	–	correct
full-range	✓	(1, 65535)	144,779,214	144,779,195	–	–	= 6542
full-range#2 hidden	✓	–	144,767,847	144,767,828	–	–	correct
rand-wide#1 hidden	✓	–	144,326,212	144,326,193	–	–	correct
rand-wide#2 hidden	✓	–	131,905,624	131,905,605	–	–	correct
rand-narrow#1 hidden	✓	–	7,944,410	7,944,391	–	–	correct
rand-narrow#2 hidden	✓	–	14,257,854	14,257,835	–	–	correct
rand-cent#1 hidden	✓	–	56,121,511	56,121,492	–	–	correct
rand-cent#2 hidden	✓	–	66,988,849	66,988,830	–	–	correct
rand-full#1 hidden	✓	–	58,886,513	58,886,494	–	–	correct
rand-full#2 hidden	✓	–	43,498,374	43,498,355	–	–	correct

The cycles displayed on the Judge system of this submission is 963M, which means your program's runtime is 963 times that of the reference program.

Speed (net cycles): you 963,552,582 vs baseline 1,000,000 → 3.98 pts (40%)

Therefore, the total score obtained is as follows:

$$20 + 10 \times (1 - \frac{963 - 1}{1600 - 1}) = 23.98$$

The final score will be rounded to the nearest integer; therefore, the final score obtained by this submission is 25 points.

Worth to notice

- In order to obtain results faster, we use a customized simulator rewritten in C (about **900x** faster), so there may be bugs or some behaviors that differ from the official simulator. Therefore, if you believe your program is fine but you still get errors on the Judge, please also contact the TA on Discord.



- If you think getting a full score on this assignment is too easy, you can try to keep optimizing your program to beat other user on the rank